

변수와 타입

체크리스트 (1/2)

- 【】 변수(let)와 상수(const)의 개념적 차이 및 적절한 사용 타이밍 구분 여부
- 【】 var의 임시 생략 배경(호이스팅 및 스코프 문제 등) 및 최신 표준 지향 여부
- 【】 5대 원시 자료형(문자열, 숫자, 논리, null, undefined)의 데이터 성격 이해 여부
- 【】 템플릿 리터럴 백틱(`)을 이용한 문자열 내 개행 및 변수 플레이스홀더(\${변수}) 조작 여부

체크리스트 (2/2)

- 【】 객체(Object)와 배열(Array)의 기본 구조 이해, 선언법 및 특정 원소 접근(인덱스/키) 가능 여부
- 【】 산술, 대입, 비교 연산자 및 삼항 연산자의 빠른 코드 압축 적용 숙지 여부
- 【】 증감 연산자(++ , --)의 전위 vs 후위 실행 순서 차이 파악 및 활용 여부
- 【】 논리 연산자의 단축 평가(Short-circuit) 특성을 이용한 기본값 설정 패턴 이해 여부

변수와 상수

변수와 상수 컴퓨터 메모리에 이름을 붙인 방을 마련하여, 필요한 데이터를 저장하고 재사용하기 위해 선언하는 식별 표지.

let 과 const

- **변수 (let)**: 선언 이후 언제든지 저장된 내부 값을 새로운 값으로 교체(재할당)할 수 있는 가변 컨테이너
- **상수 (const)**: 선언 시 반드시 초기값을 채워야 하며, 최초 할당 완료 후 값의 임의 변경(재할당)이 제한되는 안전형 고정 컨테이너

할당 및 출력 호출 시각화 예시

```
let count = 10;      // 변수 선언 및 초기값 10 대입
const limit = 100;   // 상수 선언 및 고정값 100 대입

console.log(count);  // 10 출력
console.log(limit);  // 100 출력

count = 20;          // 값 재할당 가능 (count는 20으로 갱신됨)
// limit = 200;      // 에러 발생! const 재할당 시 TypeError 송출
```

자료형(타입) - 1. 기본 자료형 (1/2)

자바스크립트의 가장 기본이 되는 독립적인 5대 원시 자료 사양:

- **Number (숫자)**: 정수와 실수를 구분하지 않고 모두 하나의 64비트 실수 포맷으로 일괄 처리함
- **Boolean (논리)**: 오직 참(true) 또는 거짓(false) 중 하나의 결과값만 지님
- **null**: 개발자가 '현재 비어 있는 값'임을 소스 상에 의도적으로 강제 표기할 때 사용
- **undefined**: 변수 공간을 선언만 해두고 값을 대입하지 않았을 때 자바스크립트 엔진이 부여하는 기본 디폴트 상태
값

자료형(타입) - 1. 기본 자료형 (2/2)

- **String (문자열)**: 텍스트 데이터를 제어하는 타입으로 총 3가지 따옴표 기호를 가짐
 - **'작은따옴표' / "큰따옴표"**: 고정형 일반 텍스트 표현
 - **`백틱 (템플릿 리터럴)`**: 중간에 물리적인 줄바꿈(개행)을 그대로 인식하며, `${변수명}` 문법을 활용해 문자열 중간에 변수 값을 부드럽게 주입 가능
- **기타 참조**: Symbol (고유 식별값), BigInt (초대형 정수)

문자열 리터럴 작성 예시

```
const name = "길동";  
const role = '프론트엔드';  
  
// 백틱을 활용한 편리한 문자열 합성 및 줄바꿈  
const profile = `회원 이름: ${name}  
담당 직무: ${role} 개발자`;  
  
console.log(profile);
```


객체와 배열 (기초) - 1. 배열

반복문이나 조건문 등 제어 흐름 학습 시 순회 타겟이 될 수 있는 기본 형태 및 인덱스/키 탐색 중심

1. 배열 (Array)

- **정의:** 여러 데이터를 순서대로 차곡차곡 나열하여 저장하는 인덱스 기반 순차 자료 목록
- **선언 및 접근:** 대괄호([])로 생성하며, 인덱스는 0번부터 시작함

```
const colors = ["빨강", "초록", "파랑"];  
console.log(colors[0]); // "빨강" 출력  
console.log(colors[2]); // "파랑" 출력  
console.log(colors.length); // 3 출력 (원소 개수)
```

객체와 배열 (기초) - 2. 객체

2. 객체 (Object)

- **정의:** 순서에 상관없이 이름(Key)과 값(Value)의 쌍들로 데이터를 매핑해 관리하는 구조
- **선언 및 접근:** 중괄호({})로 구성하며 점 표기법(.) 또는 대괄호 속성 검색법을 활용함

```
const computer = {  
  cpu: "i7",  
  ram: "16GB",  
  ssd: true  
};  
console.log(computer.cpu);    // "i7" 출력 (점 표기)  
console.log(computer["ram"]); // "16GB" 출력 (대괄호 키 탐색)
```

연산자 종류 분류

- **대입 연산자**: =(할당), +=, -= (복합 연산)
- **산술 연산자**: +, -, *, /, %(나머지 나눗셈)
- **비교 연산자**: ==(동등 비교 - 값과 타입의 완벽한 동일 여부 검증), !=(비동등 비교), >, <
- **삼항 연산자**: 조건식 ? 참인_경우_값 : 거짓인_경우_값
 - 짧은 판단에 따라 결과값을 즉시 변수에 담거나 리턴 처리할 때 매우 유용하여 현업 권장함

```
const age = 20;  
const entry = age >= 19 ? "입장 가능" : "입장 제한";  
console.log(entry); // "입장 가능" 출력
```

증감 연산자 순서의 함정

- **전위 증감 (++x):** 식 평가에 들어가기 전에 변수 값을 1 먼저 증가시킴
- **후위 증감 (x++):** 현재 평가식을 원래의 값으로 우선 연산 종결한 뒤, 최종 변수 값을 1 증가시킴

```
let a = 10;  
let b = ++a; // a를 11로 올린 뒤 b에 대입 (a=11, b=11)
```

```
let x = 10;  
let y = x++; // y에 10을 먼저 대입한 뒤 x를 11로 증가 (x=11, y=10)
```

논리 연산자 및 단축 평가

- **논리곱 (&&):** 좌변이 false면 즉시 평가를 멈추고 false를 뱉음 (우변을 무시함)
- **논리합 (||):** 좌변이 true면 즉시 평가를 멈추고 true를 뱉음 (우변을 무시함)
- **단축 평가 (Short-circuit):** 좌변에 에러가 날 수 있는 객체 검사나, 누락된 값에 대한 디폴트 세팅 시 강력하게 쓰임

```
// || 연산자를 사용한 유저명 누락 기본값 방어 패턴
let nickname = "";
let activeName = nickname || "방문객";
console.log(activeName); // ""은 Falsy이므로 우변인 "방문객"이 최종 반환됨
```